Spring'26 CPSC 323.02 Compilers & Languages
HW #5 [100 points]

Submission deadline: 11:59 PM, Sunday, May 3

*Any class content /HW shall **NOT** be made publicly accessible without the written consent of the instructor.*

Build a toy compiler front-end that will recognize and evaluate an arithmetic expression generated by the given context-free grammar G as follows, in which *int* is a terminal symbol representing integers.

$$E \rightarrow E + T \mid E - T \mid -E \mid T$$
$$T \rightarrow T * F \mid T/F \mid F$$
$$F \rightarrow int \mid (E)$$

We require the parser that will:

1. implement a calculator to compute the result of the input arithmetic expression if the input arithmetic expression is in the language of the given grammar (meaning that is a valid arithmetic expression).
2. produce an error message if its input arithmetic expression is not in the language of the given grammar (meaning that is an invalid arithmetic expression).

We also require each input arithmetic expression follows the following precedence and associativity rules:

- Unary minus (-) have precedence over all operators. In the YACC source file, you need to declare a symbolic precedence name UMINUS for unary minus and then apply it to the unary-minus production like this: "*'-' E %prec UMINUS* " in the Grammar section.
- Parentheses have precedence over the binary operators *, /, +, and -.
- * and / have precedence over the binary operators + and -.

There are **two** options for building your toy compiler.

- If you choose **Option 1**, you <u>must complete individually</u> on it.
- If you choose **Option 2**, <u>you can choose to complete it in a team of 2 persons, OR you can choose to complete it individually</u>. If you work as a team, please make sure to indicate **all** members' full names & CWID# in your submitted documents and in the Canvas comment box.

Option 1: Write a Lex-Yacc (or Flex-Bison) supported program to recognize and evaluate the input arithmetic expressions generated by the given G (on page 1). Lex is a lexer generator generating a lexer (scanner). Yacc is a LALR parser generator generating a LALR parser (a type of bottom-up parser). **Note:** In an LALR parser, left recursion is allowed in the grammar. Therefore, you may use the given grammar G (on page 1) directly.

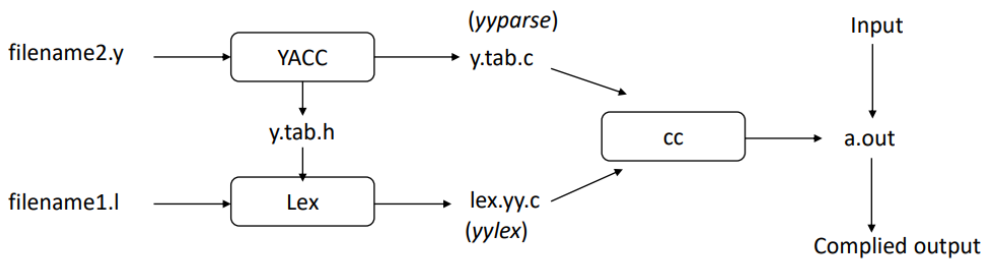

Fig. 1: The process of building a compiler front-end with Lex & Yacc

- Two starter code templates (*lex.l, yacc.y*) are provided for you, and you are strongly encouraged to use it.
- A test case sample file (*sample_test_cases.txt*) is provided to help you debug and verify your program before the final submission. **Note:** You can submit your program on Canvas **only once** for this assignment. Your final submission will be evaluated using additional test cases in addition to the provided test cases. So, please debug your program thoroughly and carefully.
- Two starter code templates and the test case sample file can be downloaded here. **Make sure to place all files in the same directory.

- **You must first install Flex and Bison to compile and test your toy compiler**. Installation tutorials can be found online for your own OS.
- After installation, compile Lex and Yacc source files using tools Flex and Bison **in order**. Then run the executable file on the sample test case file. Terminal debugging and testing are also supported in the given code templates.

Commands suitable for Unix / Linux / Windows MSYS2 Terminal

```
flex lex.l
```

bison -d yacc.y (The `-d` option tells Bison to generate both: `yacc.tab.c yacc.tab.h`)

```
gcc lex.yy,c yacc.tab.c -o your_compiler_name

./your_compiler_name < sample_test_cases.txt > test_case_output.txt
```

Option1 Submission: your final submission to Canvas must have **3** files: lex source file (lex.l)**,** yacc source file (yacc.y)**,** and a README file showing how to compile and run the program.

Option 2: Write your own program that implements a *lexer* (that can be your HW1 code) and a *predictive recursive-descent parser* (a type of *LL(1)* parser) together, to recognize and evaluate any input arithmetic expressions generated by given G (on page 1). **Notice:** *the given G is unsuitable for any type of top-down parser, because some left-recursions lead to an infinite recursion (loop), and hence the parser cannot terminate.* Therefore, we need to transform G to an equivalent grammar G' by eliminating left recursions in G. The language described by G' is the same as that of grammar *G*, and G' is unambiguous. Each choice can be made by looking at the next token in the input string. The *G'* is given below for use in your program.

$$E \rightarrow TE'$$
$$E' \rightarrow +TE' \mid -TE' \mid \varepsilon$$
$$T \rightarrow FT'$$
$$T' \rightarrow *FT' \mid /FT' \mid \varepsilon$$
$$F \rightarrow -F \mid \text{int} \mid (E)$$

The idea of implementing a *predictive recursive-descent parser* is to transform each nonterminal in the grammar G' to a parsing function that will recognize exactly that nonterminal in the input using one token of lookahead to make parsing decisions. Please write your program in C++ (preferred) or Python.

- A starter C++ code template (*starter_code.cpp*) is provided, and you are strongly encouraged to use it. You may also choose to implement in Python (but no starter code provided).
- A test case sample file (*sample_test_cases.txt*) is provided to help you debug and verify your program before the final submission. **Note:** You can submit your program on Canvas **only once** for this assignment. Your final submission will be evaluated using additional test cases in addition to the provided test cases. So, please debug your program thoroughly and carefully.
- The code template and the test case sample file can be downloaded here.

➢ Compile *starter_code.cpp* and run the executable file on the given test case file. *Terminal debugging and testing are also supported in the given code template.*

1. **Commands suitable for Windows PowerShell:**

```
g++ -std=c++17 starter_code.cpp -o your_compiler_name

Get-Content .\sample_test_cases.txt | .\your_compiler_name | Set-Content .\test_case_output.txt
```

2. **Commands suitable for Unix / Linux**

```
gcc -std=c++17 starter_code.cpp -o your_compiler_name

./your_compiler_name < sample_test_cases.txt > test_case_output.txt
```

Option2 Submission: You must submit **2** files: the source program (.cpp or .py), and a README file showing how to compile and run the program. For a teamwork, only one submission is required from one team member.

REFERENCE

- Keith Clarke, "The top-down parsing of expressions", Research Report 383, Dept of Computer Science, Queen Mary College.

- Vaughn R. Pratt, "Top-down operator precedence", Proceedings of the 1st symposium on principles of programming languages (POPL), 1973, http://dl.acm.org/citation.cfm?id=512931

- **Lex & Yacc:** Textbook Appendix B.1 and B.2

- **Bison Installation:**
  - **Bison for Windows: http://gnuwin32.sourceforge.net/packages/bison.htm**

  - **Bison for Ubuntu/Linux: https://installati.one/ubuntu/21.04/bison/**

  - **Installation videos for Lex-Yacc (or Flex-Bison):**
    - **https://www.youtube.com/watch?v=VUOapK1zKAA**
    - **https://www.youtube.com/watch?v=uPFMLstILns**